Rapport technique

VolyLegume — MaraîchèrePro

Application web de gestion d'exploitations maraîchères

Projet individuel — L3 SIO — ESMIA

Étudiant : ANDRIAMANANA Ammi Joda — SE20240348

Version 2.0 — 2 novembre 2026

Résumé

VolyLegume est une application web de gestion d'exploitations maraîchères conçue pour le contexte malgache. Elle permet à une exploitation de piloter ses parcelles, ses cultures, ses campagnes, son calendrier cultural, ses activités, ses intrants et stocks, ses traitements phytosanitaires, ses récoltes, ses lots de produits frais, ses clients, ses ventes, ses paiements et ses finances simplifiées. L'application est développée avec Laravel 13 et PHP 8.3+ (PHP 8.5 en développement), rendue côté serveur avec Blade et Tailwind CSS, et exposée partiellement via une API REST sécurisée par Sanctum.

Le présent rapport documente l'architecture, les choix techniques, les règles métier, le dispositif de sécurité, le journal d'audit, les tests et les conditions d'exploitation de la version livrée. Il met également en évidence la conformité au cahier des charges, les écarts résiduels et les perspectives d'évolution.

La version décrite est couverte par **152 tests automatisés et 544 assertions**, tous réussis. Elle comprend un contrôle d'accès centralisé par Policies Laravel, un journal d'audit transversal avec purge planifiée, un panneau d'administration de plateforme, un plan interactif des parcelles, un module de calendrier cultural, un module de fiches de culture, un suivi d'irrigation avec quotas d'eau, un historique photo des parcelles, des alertes de retard critique au-delà de 3 jours, la traçabilité publique par QR code, une chaîne commerciale complète : lot de récolte, stock de produits frais, clients, ventes multi-lignes, paiements partiels, recettes et annulation avec restitution du stock, une analyse de rentabilité par campagne, culture et parcelle, des exports CSV et des états imprimables, une supervision opérationnelle exposée par `/health`, ainsi qu'une sauvegarde et une restauration vérifiables de la base et des fichiers publics. L'application est installable comme PWA et permet de conserver localement les activités et récoltes saisies hors connexion, puis de les synchroniser sans doublon. La navigation principale reste lisible tout en donnant accès direct aux fonctions quotidiennes ; les fonctions secondaires sont regroupées dans un hub dédié, accessible en un clic.

Sommaire

1. Introduction et contexte
2. Besoins et conformité au cahier des charges
3. Architecture logicielle
4. Modèle de données
5. Sécurité et contrôle d'accès
6. Fonctionnalités métier
7. Journalisation et audit
8. API REST
9. Tests et qualité logicielle
10. Déploiement et exploitation
11. Bilan et perspectives
12. Annexes

---

1. Introduction et contexte

1.1 Contexte du projet

Le projet VolyLegume s'inscrit dans le cadre d'un projet fil rouge académique. Il répond à un besoin concret : les exploitations maraîchères de petite et moyenne taille disposent souvent de peu d'outils numériques adaptés à leur réalité quotidienne. Les registres papier, les tableurs et les échanges informels rendent difficile le suivi des parcelles, des intrants, des traitements et de la rentabilité.

L'application vise à fournir un outil simple, cohérent et traçable, utilisable par plusieurs personnes au sein d'une même exploitation. Elle doit également garantir qu'une exploitation ne puisse jamais accéder aux données d'une autre, même en manipulant les URL ou les identifiants.

Le contexte est celui de Madagascar, avec une devise unique en ariary (MGA), une localisation française et un fuseau horaire `Africa/Antananarivo`. Les données de démonstration sont fictives et ne constituent en aucun cas des prescriptions agronomiques.

1.2 Objectifs

Les objectifs poursuivis sont les suivants :

- centraliser les données de production d'une exploitation maraîchère ;
- suivre le cycle cultural, depuis la préparation du sol jusqu'à la récolte ;
- sécuriser les opérations sensibles, notamment les traitements phytosanitaires et les récoltes ;
- fournir des indicateurs de pilotage simples : marge, rendement, taux de réalisation, alertes de stock ;
- tracer les opérations pour faciliter le contrôle, la responsabilisation et la démonstration ;
- exposer une API REST permettant des intégrations futures ;
- garantir l'isolation des données entre exploitations ;
- fournir une base technique maintenable, testée et documentée.

1.3 Périmètre

Le périmètre fonctionnel couvre :

- la gestion des comptes et des rattachements à une ou plusieurs exploitations ;
- les rôles d'exploitation : administrateur, responsable, agent ;
- un rôle global d'administrateur de plateforme ;
- les parcelles et les cultures ;
- un plan interactif des parcelles, avec relief indicatif et panneau de détail ;
- les campagnes culturales et le calendrier cultural ;
- les activités de suivi, avec alertes de retard critique au-delà de 3 jours ;
- l'irrigation et le suivi des volumes d'eau par campagne ;
- les photos de parcelles et leur historique ;
- la traçabilité publique des lots par QR code ;
- les intrants, les mouvements de stock et les alertes de seuil ;
- les traitements phytosanitaires et le délai avant récolte ;
- les récoltes et les lots de produits frais ;
- les clients, les ventes multi-lignes et les paiements partiels ;
- les dépenses, recettes et indicateurs financiers simplifiés ;
- les notifications internes ;
- le journal d'audit ;
- l'API REST de consultation des campagnes et de saisie des activités.

Le périmètre exclut notamment :

- la comptabilité générale ou analytique complète ;
- la gestion de la paie ;
- la traçabilité réglementaire officielle ;
- la facturation client ;
- la gestion multi-devises ;

- les applications mobiles natives ;
- l'envoi de courriels transactionnels en production ;
- l'intégration avec des capteurs ou des stations météo.

1.4 Méthode de travail

La démarche adoptée est itérative. Elle a consisté à :

1. analyser le cahier des charges et le code existant ;
2. identifier les écarts entre les exigences et l'implémentation ;
3. renforcer les fondations techniques : Policies, audit, administration ;
4. compléter les tests automatisés ;
5. documenter l'architecture et les règles métier ;
6. vérifier la conformité par une matrice de couverture ;
7. formuler les limites et les perspectives.

Cette méthode a permis de traiter d'abord les risques structurants : autorisations, isolation multi-exploitations, traçabilité et administration. Les fonctionnalités métier déjà présentes ont été conservées et sécurisées par les Policies.

1.5 Public visé

Ce rapport s'adresse :

- à l'équipe pédagogique, pour l'évaluation technique et fonctionnelle ;
- aux développeurs qui reprendront le projet ;
- aux exploitants et responsables qui souhaitent comprendre le fonctionnement ;
- aux administrateurs chargés du déploiement.

---

2. Besoins et conformité au cahier des charges

2.1 Acteurs et rôles

L'application distingue deux niveaux de rôles.

2.1.1 Rôles d'exploitation

Le rôle est porté par le rattachement `farm_user.role`. Un utilisateur peut avoir des rôles différents selon l'exploitation.

Rôle	Libellé	Gestion des référentiels	Gestion des finances	Gestion de l'équipe	Saisie des activités
`admin`	Administrateur d'exploitation	Oui	Oui	Oui	Oui
`responsable`	Responsable / chef d'exploitation	Oui	Oui	Oui	Oui
`agent`	Agent / technicien	Non	Non	Non	Oui

Les capacités sont centralisées dans l'énumération `FarmRole`, ce qui évite de disperser des conditions dans les contrôleurs et les vues.

2.1.2 Administrateur de plateforme

Un utilisateur peut porter le booléen `is_platform_admin`. Ce rôle est transversal : il ne remplace pas les rôles d'exploitation, mais il autorise l'accès au panneau `/admin`. Il permet de :

- consulter les indicateurs globaux ;

- gérer les comptes ;
- activer ou désactiver un compte ;
- attribuer ou retirer le rôle de plateforme ;
- gérer les rattachements aux exploitations ;
- superviser les exploitations ;
- consulter le journal d'audit global.

Ce rôle répond au besoin d'administration qui n'était pas couvert par le rôle `admin` d'exploitation.

2.2 Besoins fonctionnels

Le tableau suivant synthétise les besoins fonctionnels et leur mise en œuvre.

Réf.	Besoin	Implémentation	État
BF01	Authentification	Session web + Sanctum pour l'API	Couvert
BF02	Inscription	`RegisterController`	Couvert
BF03	Déconnexion	`LogoutController`	Couvert
BF04	Multi-exploitations	`FarmContext`, `farm_user`	Couvert
BF05	Rôles et permissions	Enums + Policies	Couvert
BF06	Gestion des parcelles	CRUD `ProductionUnit`	Couvert
BF07	Gestion des cultures	CRUD `Crop` + catégories	Couvert
BF08	Fiches de culture	`CropGuide`, `CropGuideService`	Couvert
BF09	Calendrier cultural	`CalendarStep`, `CropCalendarService`	Couvert
BF10	Campagnes	CRUD `Campaign`	Couvert
BF11	Activités	CRUD et réalisation	Couvert
BF12	Intrants	CRUD `Input`	Couvert
BF13	Stock	`StockService`, mouvements	Couvert
BF14	Alertes de stock	Événement + notification + tableau de bord	Couvert
BF15	Traitements phytosanitaires	`TreatmentService`	Couvert
BF16	Délai avant récolte	Borne calculée côté serveur	Couvert
BF17	Récoltes	`HarvestService`	Couvert
BF18	Dépenses et recettes	CRUD + policies finances	Couvert
BF19	Tableau de bord	`DashboardService` + cache	Couvert
BF20	Notifications	Canal `farm` + base de données	Couvert
BF21	Journal d'audit	`AuditLog`, `AuditLogger`, `Auditable`	Couvert
BF22	Administration plateforme	`/admin` + policies	Couvert
BF23	API REST	Sanctum + ressources	Couvert
BF24	Recherche et pagination	Filtres et `paginate`	Couvert
BF25	File d'attente	`NotifyStockCritical`	Couvert
BF26	Lots de produits récoltés	`ProduceLot`, `ProduceMovement`, `ProduceStockService`	Couvert
BF27	Clients	CRUD `Customer`	Couvert
BF28	Ventes multi-lignes	`Sale`, `SaleItem`, `SaleService`	Couvert
BF29	Paiements et reste à payer	`Payment`, `SaleService::recordPayment`	Couvert
BF30	Traçabilité commerciale	Lot → récolte → campagne → vente → recette	Couvert
BF31	Plan interactif des parcelles	`FarmMapService`, vue `/plan`, relief CSS	Couvert
BF32	Navigation par hub	Menu principal réduit, `ModuleController`, vue `/modules`	Couvert

2.3 Règles de gestion principales

Les règles de gestion structurantes sont les suivantes.

RG01 — Isolation par exploitation

Toute ressource métier appartient à une exploitation. Le contexte actif est résolu côté serveur à partir des rattachements de l'utilisateur connecté. Aucun identifiant d'exploitation fourni par le client n'est accepté pour les routes web.

RG02 — Rôles

Un agent peut saisir des activités et consulter les données de son exploitation, mais ne peut pas gérer les référentiels, les finances, l'équipe ni les ajustements de stock. Un responsable ou un administrateur d'exploitation peut gérer ces éléments.

RG03 — Stock non négatif

Toute variation de stock passe par `StockService`. Le service verrouille la ligne d'intrant, calcule le solde après opération et refuse toute valeur négative. Le solde `inputs.quantity` reste cohérent avec la somme des mouvements.

RG04 — Traitement atomique

L'enregistrement d'un traitement crée dans une même transaction :

- une activité de type traitement ;
- un mouvement de stock de consommation ;
- une ligne de traitement phytosanitaire.

Si une étape échoue, l'ensemble est annulé.

RG05 — Délai avant récolte

La borne minimale de récolte est calculée côté serveur : `applied_at + pre_harvest_days × 24 heures`. La valeur est figée dans `minimum_harvest_at`. Une récolte antérieure à la borne la plus tardive applicable est refusée.

RG06 — Calendrier idempotent

La génération du calendrier ne duplique pas les activités existantes. Les activités réalisées ou annulées sont préservées ; seules les activités planifiées sont mises à jour.

RG07 — Traçabilité

Les créations, modifications et suppressions des modèles sensibles sont journalisées. Les connexions, les refus d'accès et les ressources masquées par l'isolation sont également tracés.

RG08 — Devise et unités

La devise est fixée à MGA. Les quantités de production et de stock sont exprimées en kilogrammes pour les récoltes et selon l'unité de base de l'intrant pour les stocks.

RG09 — Validation serveur

Les règles de validation sont appliquées côté serveur. Les contrôles côté client ne sont qu'une aide à la saisie.

RG10 — Administration séparée

L'administration de plateforme est distincte de l'administration d'exploitation. Un administrateur d'exploitation ne peut pas accéder au panneau global s'il n'est pas `is_platform_admin`.

RG11 — Stock de produits récoltés

Chaque récolte crée un lot unique avec un mouvement d'entrée. Toute sortie (vente, perte, don, consommation) ou correction passe par `ProduceStockService`, qui verrouille le lot et refuse un stock disponible négatif. Une vente ne peut porter que sur des lots de l'exploitation active.

RG12 — Ventes, paiements et recette

Une vente confirmée calcule son sous-total, sa remise et son total côté serveur, déduit le stock des lots et crée une recette unique. Les paiements sont plafonnés au total de la vente. L'annulation n'est possible qu'avant tout paiement : elle restaure le stock, supprime la recette et conserve l'historique des mouvements.

RG13 — Plan schématique et relief indicatif

Le plan n'utilise pas de contour géographique, faute de relevé fiable. Les parcelles sont disposées proportionnellement à leur surface dans un cadre schématique. Le relief 3D est un rendu CSS décoratif : il ne doit pas être interprété comme une mesure de pente ou de distance. Les dimensions `length_m` et `width_m` sont facultatives et servent uniquement à enrichir la fiche de la parcelle.

RG14 — Navigation lisible

Le menu principal expose les neuf entrées les plus utilisées : tableau de bord, plan, parcelles, campagnes, activités, irrigation, agenda, stock récolté et hub. Toute fonction secondaire reste accessible en un clic depuis le hub, soit deux clics depuis une page principale. Les entrées réservées aux responsables et administrateurs (équipe, journal d'audit) ne sont affichées qu'après autorisation.

2.4 Besoins non fonctionnels

Réf.	Besoin	Réponse
BNF01	Sécurité	Policies, CSRF, validation, hachage Argon2id, Sanctum
BNF02	Isolation	Contexte serveur + `BelongsToFarm`
BNF03	Traçabilité	Journal d'audit transversal
BNF04	Maintenabilité	Architecture MVC + services + Policies
BNF05	Testabilité	PHPUnit, SQLite et PostgreSQL, 152 tests
BNF06	Performance	Cache tableau de bord, index SQL, pagination
BNF07	Robustesse	Transactions, verrous pessimistes, contraintes SQL
BNF08	Portabilité	Docker, PostgreSQL, configuration `.env`
BNF09	Accessibilité	Interface Blade responsive, libellés français
BNF10	Évolutivité	API REST, services découplés

2.5 Matrice de conformité

Exigence du cahier des charges	Réalisation	Preuve
Rapport technique 15–25 pages	Ce document	Sections 1 à 12
Policies Laravel	19 Policies + trait commun	`app/Policies`
Journalisation	`audit_logs`, `AuditLogger`, `Auditable`	`app/Support`, `app/Models/Concerns`
Administration globale	Panneau `/admin`	`app/Http/Controllers/Admin`
Tests	152 tests, 544 assertions	`php artisan test`
Base de données 8–15 tables (indicatif)	26 tables métier et 9 tables techniques, périmètre détaillé assumé	`database/migrations`

Exigence du cahier des charges	Réalisation	Preuve
Factories et seeders	Factories par modèle + `DatabaseSeeder`	`database/factories`, `database/seeders`
Module personnel	Calendrier cultural paramétrable et suivi des retards	`CropCalendarService`, `/agenda`
Navigation	Menu réduit + hub des fonctions secondaires	`ModuleController`, `/modules`
Chaîne commerciale	Lots, clients, ventes, paiements, recettes	`ProduceStockService`, `SaleService`
Isolation multi-exploitations	Contexte serveur + route binding	`FarmContext`, `BelongsToFarm`
API REST	Sanctum + endpoints	`routes/api.php`

2.6 Écarts résiduels

Les écarts identifiés sont les suivants :

- l'historique Git reste limité et doit être enrichi par des commits explicites ;
- la couverture de tests peut encore être étendue aux cas limites de l'API ;
- la purge du journal d'audit est planifiée, mais dépend du cron `schedule:run` en production ;
- l'envoi de courriels réels n'est pas configuré ;
- la documentation utilisateur reste synthétique ;
- la supervision se limite aux contrôles internes de `/health` : aucune intégration Sentry ni métriques externes n'est installée ;
- la file hors ligne couvre les activités et les récoltes, sans synchronisation en arrière-plan native.

Ces écarts ne remettent pas en cause les fonctionnalités principales, mais ils constituent des axes d'amélioration.

---

3. Architecture logicielle

3.1 Vue d'ensemble

L'application suit l'architecture MVC de Laravel, complétée par des services métier et des Policies.

```
Navigateur / client API
        ↓
Routes web et API → middleware (auth, farm, admin)
        ↓
Contrôleurs → validation, autorisation, orchestration
        ↓
Services métier → règles, transactions, verrous
        ↓
Modèles Eloquent → relations, scopes, casts
        ↓
PostgreSQL (SQLite pour les tests)
```

Le journal d'audit est transversal : il est déclenché par les événements Eloquent et par les services, puis écrit dans `audit_logs`.

3.2 Organisation des dossiers

- `app/Http` : contrôleurs web, API et administration, middleware, requêtes et ressources ;
- `app/Models` : modèles Eloquent, relations, scopes et traits `Auditable` / `BelongsToFarm` ;
- `app/Services` : règles métier réutilisables ;

- `app/Policies` : décisions d'autorisation centralisées ;
- `app/Events`, `Listeners`, `Jobs`, `Notifications` : événements, réactions différées et alertes ;
- `app/Support` : contexte d'exploitation, cache, audit, exports ;
- `database` : migrations, factories et seeders ;
- `resources/views` : interface Blade et Tailwind ;
- `routes` : routes web, API et console ;
- `tests` : tests unitaires et fonctionnels.

3.3 Rôle des couches

Contrôleurs

Les contrôleurs restent minces : ils valident les entrées, autorisent l'action via les Policies, délèguent les opérations complexes aux services, puis retournent une vue ou une redirection. L'administration vérifie le rôle de plateforme avant toute action.

Services

Service	Responsabilité
`StockService`	Mouvements, verrouillage, solde, seuil critique
`TreatmentService`	Traitement atomique et borne de récolte
`HarvestService`	Contrôle du délai et enregistrement
`CropCalendarService`	Prévisualisation, génération, replanification
`CropGuideService`	Fiche de culture et étapes
`FarmMapService`	Plan, disposition des parcelles, alertes et légende
`DashboardService`	Agrégats et cache
`BackupService`	Dump, archive des fichiers et restauration vérifiée
`ProfitabilityService`	Marge, rendement et eau par campagne, culture et parcelle
`HealthService`	Contrôles base, file, sauvegarde et stockage

Modèles et Policies

Les modèles portent les relations, casts, scopes (`forFarm`, `overdue`, etc.), les traits transversaux et les contraintes applicatives. Les Policies centralisent les autorisations ; le trait `ChecksFarmRole` fournit le rôle courant, l'appartenance à l'exploitation et les capacités de gestion.

3.4 Flux d'une requête

Web — création d'une dépense : session authentifiée → `ResolveFarmContext` → `RequireActiveFarm` → validation → `ExpensePolicy::create` → insertion → `Auditable` / `AuditLogger` → invalidation du cache → redirection.

API : token Bearer → `auth:sanctum` → `ResolveApiFarmContext` avec `X-Farm-Id` → vérification du rattachement → `resolveRouteBinding` limité à l'exploitation → réponse JSON.

3.5 Choix techniques

- **Laravel 13** : routage, ORM, validation, sessions, files, notifications et tests ; configuration moderne via `bootstrap/app.php`.
- **Blade + Tailwind** : rendu serveur rapide, responsive, sans complexité d'application monopage.
- **PostgreSQL 18.6** : contraintes `CHECK`, clés étrangères composites et verrous pessimistes ; SQLite uniquement pour les tests.
- **Sanctum** : authentification par token simple pour l'API.
- **PHPUnit** : tests fonctionnels et d'intégration avec `RefreshDatabase`.

4. Modèle de données

4.1 Vue relationnelle

Le schéma comporte 26 tables métier et 9 tables techniques Laravel. Les relations structurantes sont :

- `users` N..N `farms` via `farm_user`, avec un rôle par exploitation ;
- `farms` 1..N `production_units`, `crops`, `inputs`, `customers`, `sales`, `expenses`, `revenues` ;
- `crops` 1..1 `crop_guides` et 1..N `calendar_steps`, `campaigns` ;
- `campaigns` 1..N `activities`, `phytosanitary_treatments`, `harvests`, `produce_lots`, `irrigation_events`, `expenses`, `revenues` ;
- `harvests` 1..1 `produce_lots` ; `produce_lots` 1..N `produce_movements` et `sale_items` ;
- `sales` 1..N `sale_items` et `payments`, et 1..1 `revenues` ;
- `audit_logs` et `notifications` référencent l'exploitation et, lorsque disponible, l'utilisateur.

4.2 Tables métier

Table	Rôle	Points clés
`users`	Comptes	`is_active`, `is_platform_admin`
`farms`	Exploitations	responsable, statut, devise MGA
`farm_user`	Rattachement	rôle, actif, unicité `(farm_id, user_id)`
`production_units`	Parcelles	code unique par exploitation, surface, plan
`crop_categories`	Catégories	référentiel global, ordre, activation
`crops`	Cultures	variété, cycle, densité, rendement
`crop_guides`	Fiches culturales	relation 1–1 avec la culture
`calendar_steps`	Modèle de calendrier	décalage, type d'activité, position
`campaigns`	Campagnes	parcelle, culture, dates, quota d'eau
`activities`	Activités	prévue/réalisée, auteur, retard
`inputs`	Intrants	unité de base, seuil, délai par défaut
`stock_movements`	Mouvements d'intrants	type, variation signée, correction
`phytosanitary_treatments`	Traitements	dose, délai, borne de récolte
`harvests`	Récoltes	quantité, qualité, contrôle du délai
`produce_lots`	Lots de produits	quantité disponible, QR, jeton
`produce_movements`	Mouvements de lots	vente, perte, don, retour, ajustement
`customers`	Clients	code unique par exploitation
`sales`	Ventes	montants calculés, statut, annulation
`sale_items`	Lignes de vente	lot, quantité, prix, total
`payments`	Paiements	plafonnés au total de la vente
`expenses`	Dépenses	campagne facultative, référence
`revenues`	Recettes	campagne, récolte ou vente facultative
`irrigation_events`	Irrigations	volume, durée, méthode, source
`production_unit_photos`	Photos	parcelle, campagne ou activité facultative
`audit_logs`	Journal d'audit	acteur, action, valeurs avant/après
`notifications`	Notifications	destinataire et contexte d'exploitation

4.3 Contraintes d'intégrité

Les migrations ajoutent des contraintes SQL lorsque le driver est PostgreSQL :

- montants, surfaces, volumes, durées et quantités strictement positifs ;
- statuts et types limités à des listes fermées ;
- signe des mouvements de stock cohérent avec le type ;
- stock d'intrant et stock de lot jamais négatifs ;
- stock disponible d'un lot compris entre zéro et la quantité initiale ;
- borne de récolte postérieure ou égale à l'application du traitement ;
- unité de récolte limitée à `kg` et devise à `MGA` ;
- paiement plafonné au total de la vente ;
- unicité du code de parcelle, du lot, de la référence de vente et de la référence cliente hors ligne par exploitation.

Les services vérifient en plus que le client, les lots, la campagne et la vente appartiennent à l'exploitation active.

4.4 Index et performance

Les index couvrent les filtres fréquents : `(farm_id, status)` sur les parcelles, `(farm_id, planned_date)` sur les activités, `(farm_id, input_id, occurred_at)` sur les mouvements, `(farm_id, campaign_id, applied_at)` sur les traitements, `(farm_id, campaign_id, harvested_at)` sur les récoltes, `(farm_id, expense_date)` et `(farm_id, revenue_date)` sur les finances, `(farm_id, sale_date)` et `(farm_id, payment_status)` sur les ventes, `(farm_id, paid_at)` sur les paiements et `(farm_id, created_at)` sur l'audit.

Les listes sont paginées par 20 éléments. Le tableau de bord est mis en cache dix minutes avec invalidation par version d'exploitation.

---

5. Sécurité et contrôle d'accès

5.1 Authentification

Le web utilise les sessions Laravel et le middleware `auth`. Les mots de passe sont hachés avec Argon2id lorsque la configuration le permet ; les anciens hachages bcrypt restent vérifiables et sont rehachés à la première connexion réussie.

L'API utilise Sanctum : le client obtient un token via `POST /api/login`, puis l'envoie dans `Authorization: Bearer <token>`. Le token ne porte pas de contexte d'exploitation ; celui-ci est fourni par `X-Farm-Id` et vérifié côté serveur.

5.2 Contexte d'exploitation

`FarmContext` conserve l'exploitation active et le rôle courant. Pour le web, `ResolveFarmContext` lit les rattachements actifs, privilégie l'exploitation mémorisée en session, sinon la première disponible, puis alimente le contexte. Pour l'API, `ResolveApiFarmContext` valide l'en-tête `X-Farm-Id`.

Aucun identifiant d'exploitation n'est accepté directement dans les URL web pour choisir le contexte. Les routes d'administration sont l'exception contrôlée : elles opèrent sur toutes les exploitations et sont réservées aux administrateurs de plateforme.

5.3 Policies

Les Policies sont déclarées dans `app/Policies` et découvertes automatiquement. Elles distinguent la consultation, la gestion des référentiels, les finances, l'équipe et l'administration de plateforme. Le trait `ChecksFarmRole` fournit le rôle courant, l'appartenance à l'exploitation et les capacités de gestion.

Modèles principaux couverts : `Farm`, `FarmUser`, `ProductionUnit`, `Crop`, `CalendarStep`, `Campaign`, `Activity`, `Input`, `StockMovement`, `PhytosanitaryTreatment`, `Harvest`, `ProduceLot`, `Customer`, `Sale`, `Payment`, `Expense`, `Revenue`, `AuditLog` et `User`.

5.4 Administrateur de plateforme

Le rôle de plateforme est accordé par `is_platform_admin`. Un `Gate::before` global autorise les abilities pour cet utilisateur, complété par des contrôles métier dans l'administration :

- interdiction de désactiver son propre compte ;
- interdiction de retirer son propre rôle de plateforme ;
- interdiction de retirer le dernier administrateur de plateforme ;
- interdiction de retirer le rôle Responsable au responsable principal d'une exploitation.

5.5 Isolation multi-exploitations

L'isolation repose sur trois niveaux :

1. **Contexte** : résolu côté serveur depuis un rattachement actif ;
2. **Requêtes** : trait `BelongsToFarm`, scope `forFarm` et `resolveRouteBinding` renvoyant 404 hors exploitation ;
3. **Contraintes** : clés étrangères composites `(farm_id, id)` empêchant les références croisées.

Cette défense en profondeur couvre les URL, les identifiants, les listes, les exports, les notifications et les jobs.

5.6 Protections web et journalisation des refus

Validation systématique via `$request->validate` ou `FormRequest`, CSRF, échappement Blade, `$fillable` limité, requêtes paramétrées, cookies chiffrés, limitation de débit API et messages 403/404 génériques.

`DeniedAccessLogger` journalise les `AuthorizationException`, `AccessDeniedHttpException` et `ModelNotFoundException` avec méthode, chemin, route, statut et utilisateur. Les requêtes anonymes ne sont pas journalisées.

---

6. Fonctionnalités métier

6.1 Parcelles, cultures et fiches

Les parcelles portent un code unique par exploitation, une surface, un emplacement et un statut. Les dimensions `length_m` et `width_m` sont facultatives et enrichissent la fiche et le plan sans imposer une géométrie exacte.

Les cultures sont des référentiels propres à l'exploitation, associés à une catégorie et à une fiche de culture. La fiche est générée à partir d'étapes communes, d'étapes de catégorie et de points spécifiques à l'espèce ; elle reste qualitative et ne remplace pas une recommandation locale.

6.2 Calendrier cultural — module personnel

Le calendrier cultural est le module personnel retenu. Chaque étape possède un décalage en jours (`offset_days`) ; la date planifiée est calculée depuis la date de référence de la campagne. La génération est idempotente : les activités planifiées sont mises à jour, les activités réalisées ou annulées sont préservées. La replanification ne touche que les activités planifiées.

Le service retourne un bilan `{ created, updated, preserved }` et évite les doublons. Le suivi des retards complète le module : une activité planifiée dépassée est signalée, puis une alerte critique est émise au-delà de trois jours.

6.3 Campagnes et activités

Une campagne associe une parcelle, une culture, une date de début, une fin prévue, une surface cultivée, un quota d'eau et un statut (`planned` , `ongoing` , `suspended` , `completed` , `cancelled`). La date de référence peut être un semis ou une plantation.

Les activités couvrent la préparation du sol, le semis, le repiquage, la fertilisation, le traitement, l'irrigation, le désherbage, la récolte, l'observation et l'entretien. Elles peuvent être planifiées, réalisées ou annulées, assignées à un utilisateur et exécutées par un autre. Le scope `overdue` détecte les retards.

6.4 Intrants et stocks

Les intrants sont catégorisés (semences, engrais, phytosanitaires, carburant, consommables) et possèdent une unité de base, une quantité courante, un seuil minimal, un délai avant récolte par défaut et une source de délai.

Toute variation passe par `StockService` : stock initial, entrée, consommation, sortie ou ajustement signé. Le service arrondit à trois décimales, refuse une variation nulle, verrouille la ligne, refuse un solde négatif, met à jour `critical_notified_at` , déclenche `StockCritical` au passage sous le seuil et invalide le cache du tableau de bord.

6.5 Traitements et récoltes

`TreatmentService` vérifie la quantité positive, le délai non négatif, l'appartenance de l'intrant et de la campagne à la même exploitation et le caractère utilisable du produit. Il crée ensuite, dans une transaction :

1. une activité réalisée ;
2. un mouvement de stock de consommation ;
3. le traitement avec sa borne de récolte.

La borne minimale est `applied_at + pre_harvest_days × 24 heures` , figée dans `minimum_harvest_at` avec snapshot de la source du délai.

`HarvestService` applique la borne la plus tardive des traitements appliqués au plus tard à la date de récolte. Une récolte antérieure est refusée par `HarvestTooEarlyException` . Chaque récolte crée automatiquement un lot `ProduceLot` et un mouvement d'entrée, ce qui permet de reconstituer la traçabilité jusqu'à la vente.

6.6 Lots, clients, ventes et paiements

Le stock de produits frais est distinct du stock d'intrants. Il est porté par les lots et leurs mouvements : entrée à la récolte, sortie à la vente, perte, don, consommation, retour ou ajustement. `ProduceStockService` verrouille le lot, refuse un disponible négatif ou supérieur à la quantité initiale et passe le lot à `depleted` lorsque le solde atteint zéro.

Les clients sont gérés par exploitation avec un code unique, un type, des coordonnées et un statut. Une vente comporte une ou plusieurs lignes rattachées à des lots. `SaleService` vérifie l'appartenance des lots, verrouille les lots, refuse une quantité supérieure au stock, calcule le sous-total, la remise et le total, crée les lignes, les mouvements de sortie et la recette, puis génère une référence unique.

Les paiements peuvent être partiels et leur cumul ne peut pas dépasser le total ; le statut passe de `unpaid` à `partial` puis `paid` . Une vente n'est annulable qu'avant tout paiement : le stock est restauré, la recette supprimée et la vente marquée `cancelled` .

6.7 Finances et indicateurs

Les dépenses et recettes sont catégorisées et peuvent être rattachées à une campagne ; une recette peut aussi l'être à une récolte ou à une vente. Le tableau de bord présente le total des dépenses, le total des recettes, la marge simplifiée, les campagnes actives, les parcelles, les intrants sous seuil, les activités en

retard, le taux de réalisation, la quantité récoltée, le rendement kg/ha, le stock de produits frais, les ventes de la période et le reste à encaisser.

`DashboardService` calcule les agrégats à la demande et les met en cache dix minutes. La clé inclut l'exploitation, une version de cache par exploitation et un hash des filtres. `DashboardCache::forget` incrémente la version, ce qui invalide toutes les combinaisons.

6.8 Irrigation, photos, alertes et traçabilité

`IrrigationEvent` enregistre la date, le volume en mètres cubes, la durée, la méthode, la source et l'utilisateur. La campagne et la parcelle doivent appartenir à l'exploitation active. `IrrigationService` produit le volume total, le nombre d'arrosages, la durée cumulée, la densité en m³/m² et l'état par rapport au quota : `ok`, `approaching` ou `exceeded`. Le seuil d'alerte est de 80 % par défaut.

Une parcelle peut recevoir une photo principale et un historique de photos, liées facultativement à une campagne ou une activité, avec légende et date. Les fichiers sont stockés sur le disque `public` et restent isolés par exploitation.

Les activités critiques exposent leur conséquence métier et l'action attendue ; `overdue_notified_at` conserve la date de première notification. `QrCodeService` génère un QR code scannable ; chaque lot possède un `trace_token` opaque et un fichier `qr_code_path`. La page `/traceabilite/{token}` présente la parcelle, la campagne, la culture, les activités, les traitements et la récolte, sans authentification. Le contenu est déclaratif et ne constitue pas une certification officielle.

6.9 Notifications, plan et navigation

L'événement `StockCritical` est traité par un listener qui envoie une notification sur le canal `farm`, stockée en base et filtrée par exploitation. Le traitement peut être différé via la file d'attente.

Le plan interactif `/plan` est une fonctionnalité complémentaire : `FarmMapService` charge les parcelles, campagnes actives, activités planifiées, récoltes récentes et traitements, puis produit une disposition proportionnelle aux surfaces, une couleur stable par culture, les alertes par parcelle et les prochaines activités. Le relief 3D est décoratif et ne doit pas être interprété comme une mesure de pente ou de distance.

Le menu principal regroupe neuf entrées (tableau de bord, plan, parcelles, campagnes, activités, irrigation, agenda, stock récolté, hub). La page `/modules` regroupe les fonctions secondaires par domaine ; les liens vers l'équipe et le journal d'audit ne sont ajoutés que si les Policies l'autorisent. Le parcours maximal est de deux clics pour une fonction secondaire.

---

7. Journalisation et audit

7.1 Objectifs et modèle

Le journal d'audit permet de savoir qui a fait quoi et quand, de reconstituer une opération sensible, de détecter les accès non autorisés et de conserver les valeurs avant/après.

Chaque ligne de `audit_logs` contient `farm_id`, `user_id`, `action`, `description`, `auditable_type`, `auditable_id`, `old_values`, `new_values`, `context`, `ip_address`, `user_agent` et `created_at`. Le modèle fournit les casts JSON, les relations `farm`, `user`, `auditable`, les scopes `forFarm` et `search`, ainsi qu'un libellé d'action lisible.

7.2 Trait `Auditable` et `AuditLogger`

Les modèles `Farm`, `FarmUser`, `ProductionUnit`, `Crop`, `CalendarStep`, `Campaign`, `Activity`, `Input`, `StockMovement`, `PhytosanitaryTreatment`, `Harvest`, `Expense`, `Revenue`, `Customer`, `ProduceLot`,

`ProduceMovement`, `Sale`, `SaleItem` et `Payment` utilisent le trait `Auditable`, qui écoute `created`, `updated` et `deleted`.

`AuditLogger` déduit l'utilisateur depuis la requête ou `Auth`, déduit l'exploitation depuis le sujet ou le contexte, exclut les champs sensibles (`password`, `remember_token`, timestamps), limite la taille des chaînes et convertit une erreur d'écriture en avertissement applicatif. Le journal ne doit jamais interrompre une opération métier.

7.3 Événements et consultation

Catégorie	Exemples
Création	`farm.created`, `campaign.created`, `expense.created`
Modification	`crop.updated`, `input.updated`
Suppression	`harvest.deleted`, `activity.deleted`
Authentification	`auth.login`, `auth.login_failed`, `auth.api_login`
Refus	`access.denied`
Administration	`admin.user.created`, `admin.user.deactivated`, `admin.membership.updated`, `admin.farm.status`

Deux écrans sont disponibles : `/journal` pour l'exploitation active (responsables et administrateurs) et `/admin/journal` pour la vue transversale (administrateurs de plateforme). Les deux permettent la recherche, la pagination et l'affichage du contexte.

7.4 Rétention

La commande `audit:prune` applique une politique de rétention et est planifiée quotidiennement à 02:30 avec `withoutOverlapping`. Les actions sensibles sont conservées plus longtemps ; la durée reste configurable. Le journal ne stocke ni valeurs binaires ni pièces jointes et ne remplace pas une sauvegarde.

---

8. API REST

8.1 Principes

L'API est exposée sous `/api`. Elle utilise :

- Sanctum pour l'authentification ;
- `X-Farm-Id` pour le contexte ;
- des ressources JSON pour la sérialisation ;
- des codes HTTP standards ;
- une limitation de débit.

8.2 Endpoints

Méthode	Route	Description	Auth
POST	`/api/login`	Obtenir un token	Non
POST	`/api/logout`	Révoquer le token	Oui
GET	`/api/campaigns`	Lister les campagnes	Oui
GET	`/api/campaigns/{campaign}`	Détail d'une campagne	Oui
POST	`/api/activities`	Créer une activité	Oui

8.3 Contexte `X-Farm-Id`

Le client doit fournir l'exploitation cible. Le middleware vérifie :

- que l'utilisateur est authentifié ;
- que le rattachement existe ;
- que le rattachement est actif ;
- que l'exploitation existe.

Sinon, une réponse 403 est renvoyée.

8.4 Réponses et erreurs

Les réponses de liste incluent des en-têtes de pagination. Les erreurs suivent les codes HTTP :

- 200 : succès ;
- 201 : création ;
- 401 : non authentifié ;
- 403 : non autorisé ou contexte absent ;
- 404 : ressource introuvable ou hors exploitation ;
- 422 : validation échouée ;
- 429 : limite de débit atteinte.

8.5 Sérialisation

Les ressources `CampaignResource` et `ActivityResource` exposent uniquement les champs utiles. Elles évitent d'exposer les relations internes ou les données sensibles.

8.6 Évolutions possibles

- endpoints de consultation des parcelles et intrants ;
- endpoints de traitement et de récolte ;
- pagination par curseur ;
- versionnement `/api/v1` ;
- documentation OpenAPI ;
- webhooks.

---

9. Tests et qualité logicielle

9.1 Stratégie

La stratégie combine tests unitaires, tests fonctionnels HTTP, tests d'intégration des services et de la base, tests d'isolation multi-exploitations, tests d'autorisation, tests d'audit et tests d'administration. Les tests utilisent `RefreshDatabase` et SQLite en mémoire pour être rapides et isolés.

9.2 Organisation

Les tests sont regroupés dans `tests/Feature` par domaine : authentification et API, administration, audit et rétention, sauvegarde/restauration, calendrier, chaîne commerciale, catalogue cultural, cache et tableau de bord, exports, isolation, plan, finances, santé, irrigation, navigation, notifications, synchronisation hors ligne, retards, mots de passe, Policies, photos, rentabilité, ressources, stock, équipe, traçabilité QR, traitements et récoltes. `tests/Unit` contient les tests unitaires isolés.

9.3 Couverture fonctionnelle

Domaine	Tests représentatifs
Authentification et API	`AuthenticationTest`, `ApiAuthTest`, `ApiActivitiesTest`, `ApiCampaignsTest`
Rôles et Policies	`PolicyAuthorizationTest`, `FinanceAuthorizationTest`, `TeamManagementTest`

Domaine	Tests représentatifs
Isolation	`FarmIsolationTest`, `NotificationFarmScopeTest`
Stock	`StockManagementTest`, dont recharge sous verrou
Traitement et récolte	`TreatmentAndHarvestTest`
Calendrier	`CalendarTest`, `CalendarGenerationTest`
Chaîne commerciale	`CommercialChainTest`
Rentabilité, exports, sauvegarde	`ProfitabilityTest`, `ExportTest`, `BackupRestoreTest`
Supervision et audit	`HealthCheckTest`, `AuditLogTest`, `AuditRetentionTest`
Hors ligne et notifications	`OfflineSyncTest`, `NotificationFarmScopeTest`
Plan et navigation	`FarmMapTest`, `ModuleNavigationTest`
Irrigation, photos, retards, QR	`IrrigationTest`, `ProductionUnitPhotoTest`, `OverdueActivityAlertTest`, `TraceabilityQrTest`
Administration	`AdminPanelTest`

9.4 Résultats

```
php artisan test --compact
Tests:    152 passed (544 assertions)
```

Les tests vérifient notamment l'accès interdit aux agents pour les référentiels et finances, l'accès autorisé des responsables, l'isolation des ressources, la journalisation des refus et des opérations métier, l'administration plateforme, le blocage des récoltes trop précoces, le stock non négatif, la génération idempotente du calendrier, la création automatique du lot, la vente avec paiements partiels et annulation, le refus d'une vente supérieure au stock ou d'un lot étranger, le plan et ses alertes, le hub de modules, l'authentification API, le hachage des mots de passe et le cycle de sauvegarde/restauration.

9.5 Limites

SQLite ne reproduit pas toutes les contraintes PostgreSQL, notamment certains `CHECK`. Les cas limites de concurrence, les tests navigateur end-to-end, la charge et les performances ne sont pas couverts. La file est testée fonctionnellement, pas en conditions de production. Le formatage est vérifié avec Laravel Pint.

---

10. Déploiement et exploitation

10.1 Prérequis et installation

Prérequis : PHP 8.3+ (8.5 en développement), Composer, Node.js/npm, PostgreSQL 18.6 et l'extension `pdo_pgsql`.

```
composer install
cp .env.example .env
php artisan key:generate
npm install && npm run build
php artisan migrate --seed
php artisan serve
```

10.2 Docker et environnement

Un `docker-compose.yml` fournit PostgreSQL sur le port `5433` pour éviter les conflits avec une instance locale. `.env.example` pointe vers `127.0.0.1:5433`, la base `volylegume` et l'utilisateur `volylegume`.

Variables critiques : `APP_ENV`, `APP_DEBUG`, `APP_KEY`, `APP_URL`, `APP_TIMEZONE`, `APP_LOCALE`, `DB_*`, `SESSION_DRIVER`, `CACHE_STORE`, `QUEUE_CONNECTION`, `HASH_DRIVER`, `ARGON_*`, `BCRYPT_ROUNDS` et `MAIL_*`. En production : `APP_DEBUG=false`, HTTPS obligatoire, cookies sécurisés, secrets hors dépôt et sauvegardes régulières.

10.3 Migrations, seeders et comptes

```
php artisan migrate:fresh --seed
```

Le seeder crée deux exploitations (Ferme Ambohipo et Exploitation Itasy), des parcelles, cultures, campagnes, activités, intrants avec alertes, traitements, récoltes, lots, clients, ventes, paiements et comptes de démonstration.

Compte	Rôle	Exploitation
`responsable@volylegume.mg`	Responsable	Ferme Ambohipo
`agent+ambohipo@volylegume.mg`	Agent	Ferme Ambohipo
`admin@volylegume.mg`	Admin plateforme + Admin	Toutes
`responsable.itasy@volylegume.mg`	Responsable	Itasy
`agent+analavory@volylegume.mg`	Agent	Itasy

Mot de passe commun : `password`.

10.4 File, cache et sessions

La file, le cache et les sessions utilisent la base de données. Le worker se lance avec `php artisan queue:work` et doit être maintenu par un superviseur en production. Les notifications de stock critique peuvent être différées.

10.5 Sauvegarde et restauration

```
php artisan backup:create
php artisan backup:restore storage/backups/20261026-153000 --force
```

`backup:create` produit `database.dump` (ou `database.sqlite`), `storage.zip` et `manifest.json` avec pilote, empreintes SHA-256 et nombre de lignes par table. `backup:restore` vérifie les empreintes et les compteurs avant de valider la restauration ; une sauvegarde absente ou altérée est refusée.

Pour une cohérence parfaite, l'application est placée en maintenance, le worker arrêté, puis les caches vidés après restauration. Les variables d'environnement et les journaux ne sont pas inclus et doivent être sauvegardés séparément. Le test `BackupRestoreTest` automatise le cycle sur SQLite et PostgreSQL.

10.6 Supervision

`GET /health` vérifie la base, la file (`jobs`, `failed_jobs`), l'âge de la dernière sauvegarde et l'écriture dans le stockage. La réponse contient `status`, `checked_at` et le détail des contrôles ; elle renvoie `200` pour `ok` ou `degraded`, et `503` si un contrôle est `down`. L'endpoint peut être protégé par `HEALTH_TOKEN` et `X-Health-Token`.

La commande `php artisan system:health` réutilise le service avec une sortie lisible ou `--json`, et un code retour `1` lorsque l'état est `down`. Une carte « Supervision opérationnelle » synthétise ces contrôles dans le

tableau de bord administrateur. La supervision externe (Sentry, métriques Prometheus, alertes 5xx) reste à intégrer.

10.7 Rentabilité, exports et mode hors ligne

`ProfitabilityService` agrège par campagne, culture et parcelle les dépenses, recettes, marges, marge au m², rendements, volumes d'eau et budget consommé. L'écran `/rentabilite` accepte les filtres `from`, `to` et `campaign_id` ; `/rentabilite/impression` produit une vue imprimable. Les exports CSV (`campagnes`, `stocks`, `traitements`, `ventes`, `finances`, `rentabilite`) sont streamés en UTF-8 avec BOM, séparés par des points-virgules et protégés contre les formules de tableur.

Le mode hors ligne repose sur un manifeste PWA, un service worker qui précache `offline.html` et les assets, une file `localStorage` par utilisateur pour les activités et récoltes, et des endpoints `/hors-ligne/activites` et `/hors-ligne/recoltes` idempotents grâce à `client_reference` et à l'index unique `(farm_id, client_reference)` . La synchronisation est relancée au retour du réseau, à l'ouverture de l'application et périodiquement ; les `422` sont des rejets de validation, les erreurs réseau ou `5xx` sont réessayées. Les pages authentifiées ne sont pas mises en cache par le service worker.

10.8 Sécurité de production

Désactiver l'inscription publique si nécessaire, restreindre `/admin` au rôle de plateforme, forcer HTTPS et HSTS, configurer un pare-feu, limiter l'accès à la base, mettre à jour les dépendances, auditer les comptes de plateforme, définir la rétention du journal et sauvegarder avant chaque migration.

---

11. Bilan et perspectives

11.1 Réalisations

La version livrée apporte une architecture MVC claire avec services métier, 19 Policies, un rôle global d'administrateur de plateforme, un journal d'audit transversal avec purge planifiée, une isolation multi-exploitations à trois niveaux, un plan interactif avec relief CSS indicatif, l'irrigation et les quotas d'eau, les photos de parcelles, les alertes critiques de retard, la traçabilité publique par QR code, le calendrier cultural idempotent comme module personnel, les fiches de culture, une navigation lisible avec hub, une gestion de stock robuste, le blocage serveur des récoltes précoces, une chaîne commerciale complète, un tableau de bord mis en cache, une API REST sécurisée, une sauvegarde/restauration vérifiable, l'analyse de rentabilité, les exports CSV, la supervision `/health`, la PWA hors ligne et 152 tests automatisés.

11.2 Apports techniques

Le projet démontre la maîtrise de Laravel 13 et PHP 8.3+ (8.5 en développement), Eloquent et ses relations, les Policies et Gates, les transactions et verrous pessimistes, les événements et listeners, les files d'attente et notifications, PHPUnit, la sécurité applicative, la modélisation de base de données et la documentation technique.

11.3 Limites

- L'historique Git doit être enrichi.
- La couverture de tests peut être améliorée sur les cas limites et l'API.
- La purge de l'audit dépend du cron de production.
- L'interface n'a pas de tests end-to-end.
- Le plan est schématique : ni contours SIG, ni fond cartographique, ni calcul de pente.
- La supervision se limite aux contrôles internes, sans Sentry ni métriques externes.
- La file hors ligne ne couvre que les activités et les récoltes.
- La documentation utilisateur reste synthétique.

- L'API ne couvre qu'une partie du domaine.
- Les contraintes SQL fortes ne s'appliquent qu'à PostgreSQL.

11.4 Perspectives

Court terme : commits structurés, tests API complémentaires, restaurations périodiques sur environnement isolé, écran de détail du journal, tests navigateur sur la file hors ligne.

Moyen terme : étendre la file hors ligne aux traitements, ventes et mouvements de stock ; archiver le journal au-delà de la purge ; compléter l'API et sa documentation OpenAPI ; brancher Sentry ou Prometheus sur `/health` ; ajouter des tests end-to-end.

Long terme : synchronisation native en arrière-plan et application mobile, météo, recommandations agronomiques contextualisées, multi-devises, comptabilité analytique, traçabilité réglementaire et tableaux de bord avancés.

11.5 Conclusion

VolyLegume répond aux besoins structurants d'une exploitation maraîchère : suivi de production, sécurité des accès, isolation des données, traçabilité et pilotage. Les renforcements apportés — Policies, audit, administration plateforme, rentabilité, exports, supervision, PWA hors ligne et tests — consolident la conformité au cahier des charges et la qualité de la base technique.

Les limites identifiées concernent surtout l'exploitation et l'industrialisation : historique Git, supervision externe, tests end-to-end et couverture hors ligne partielle. Elles n'empêchent pas la démonstration fonctionnelle et constituent une feuille de route claire.

---

12. Annexes

Annexe A — Arborescence simplifiée

```
VolyLegume/
├── app/                Console, Enums, Events, Exceptions, Http, Jobs, Listeners,
│                       Models, Notifications, Policies, Providers, Services, Support
├── database/           factories, migrations, seeders
├── public/             manifest.webmanifest, sw.js, offline.html
├── resources/          js, views
├── routes/             web.php, api.php, console.php
├── tests/              Feature, Unit
├── docker/             configuration PostgreSQL
├── docker-compose.yml
├── composer.json
├── package.json
└── README.md
```

Annexe B — Matrice rôles / actions

Action	Agent	Responsable	Admin exploitation	Admin plateforme
Voir tableau de bord	Oui	Oui	Oui	Oui
Voir parcelles	Oui	Oui	Oui	Oui
Créer/modifier parcelles	Non	Oui	Oui	Oui
Voir cultures	Oui	Oui	Oui	Oui
Créer/modifier cultures	Non	Oui	Oui	Oui

Action	Agent	Responsable	Admin exploitation	Admin plateforme
Voir campagnes	Oui	Oui	Oui	Oui
Créer/modifier campagnes	Non	Oui	Oui	Oui
Voir activités	Oui	Oui	Oui	Oui
Créer activités	Oui	Oui	Oui	Oui
Modifier/supprimer activités	Non	Oui	Oui	Oui
Voir stocks	Oui	Oui	Oui	Oui
Entrées/sorties stock	Oui	Oui	Oui	Oui
Ajuster stock	Non	Oui	Oui	Oui
Voir traitements	Oui	Oui	Oui	Oui
Créer traitements	Non	Oui	Oui	Oui
Voir récoltes	Oui	Oui	Oui	Oui
Créer récoltes	Oui	Oui	Oui	Oui
Supprimer récoltes	Non	Oui	Oui	Oui
Voir stock récolté	Oui	Oui	Oui	Oui
Ajuster un lot	Non	Oui	Oui	Oui
Voir clients	Oui	Oui	Oui	Oui
Gérer clients	Non	Oui	Oui	Oui
Voir ventes	Non	Oui	Oui	Oui
Créer/annuler ventes	Non	Oui	Oui	Oui
Encaisser un paiement	Non	Oui	Oui	Oui
Voir finances	Non	Oui	Oui	Oui
Gérer finances	Non	Oui	Oui	Oui
Gérer équipe	Non	Oui	Oui	Oui
Voir journal exploitation	Non	Oui	Oui	Oui
Accéder `/admin`	Non	Non	Non	Oui

Annexe C — Commandes utiles

```
# Installation
composer install
npm install
cp .env.example .env
php artisan key:generate
npm run build

# Base de données
php artisan migrate
php artisan migrate:fresh --seed
php artisan produce:backfill-lots

# Développement
php artisan serve
npm run dev
php artisan queue:work

# Qualité
php artisan test
vendor/bin/pint

# Docker PostgreSQL
docker compose up -d
```

Annexe D — Glossaire

Terme	Définition
Campagne	Cycle de production d'une culture sur une parcelle
Client	Acheteur identifié ou occasionnel rattaché à une exploitation
Contexte d'exploitation	Exploitation active pour la requête
Délai avant récolte	Nombre de jours entre un traitement et la récolte autorisée
Intrant	Produit consommé par l'exploitation (semence, engrais, etc.)
Lot de récolte	Quantité de produits frais issue d'une récolte, suivie en stock
Parcelle	Unité physique de production
Paiement	Encaissement partiel ou total rattaché à une vente
Plan interactif	Vue schématique des parcelles proportionnelle aux surfaces, avec relief indicatif
Policy	Classe Laravel centralisant une décision d'autorisation
Rattachement	Lien entre un utilisateur et une exploitation, avec un rôle
Tenant	Exploitation isolée dans un contexte multi-exploitations
Traitement phytosanitaire	Application d'un produit sur une culture
Vente	Transaction commerciale composée de lignes de lots et d'un montant total

Annexe E — Références

- Documentation Laravel 13 : https://laravel.com/docs
- Documentation PHPUnit : https://phpunit.de/documentation.html
- Documentation PostgreSQL : https://www.postgresql.org/docs/
- Documentation Sanctum : https://laravel.com/docs/sanctum
- Documentation Tailwind CSS : https://tailwindcss.com/docs
- OWASP Top 10 : https://owasp.org/www-project-top-ten/